

SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3-Reverse Engineering

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192524358
Name	Shaik. Abdul Razak

COURSE OUTCOME COVERED

CO5: Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d) classifying the

Real-Time Emotion Detection System — processing pipeline



agent type (e.g., simple reflex, model-based) with justification.

Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

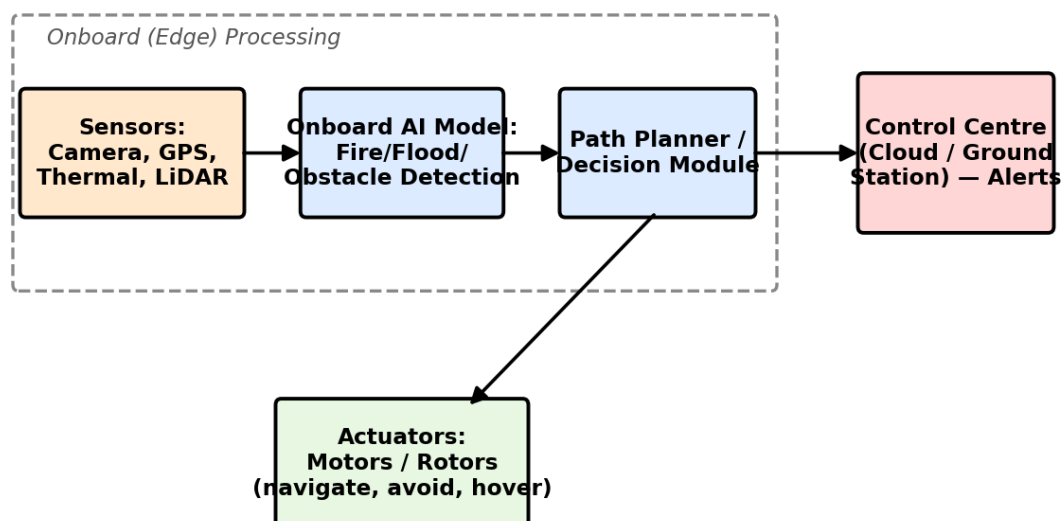
Given facts: fever = yes, cough = yes, body_ache = yes

Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.

Explanation generated by the shell: “Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza.”

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Autonomous Drone — Disaster Detection (Edge AI) system architecture



Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores,

and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	10
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	10
Identification of Underlying Technique/Algorithm/Reas	Correctly reverse-engineers the technique, algorithm, or	Correctly identifies the technique/mechanism	Identification is only partially correct or lacks	Technique/mechanism is incorrectly identified or not addressed.	10

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
oning Mechanism	reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	with adequate, mostly accurate justification.	sufficient supporting evidence.		
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	10
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner)	All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner.	Most sub-parts are addressed clearly with minor gaps in organisation or completeness.	Some sub-parts are missing or the presentation is unclear/poorly organised.	Response is incomplete, disorganised, or largely missing.	10
				Total	50

Q1. Flight-Booking Chatbot

(a) PEAS Description

PEAS Component	Description
Performance Measure	Correct flight booking, relevant flight recommendations, user satisfaction, fast response, and successful completion of booking.
Environment	Users, airline databases, flight schedules, prices, availability, booking/payment systems.
Actuators	Chat messages, flight recommendations, booking confirmation, questions/requests for missing information.
Sensors	User's text messages, stated destination, date, time preference, airline preference, and responses to chatbot questions.

b) Type of Agent

The chatbot is mainly a model-based goal-based agent.

- It has a goal of helping the user complete a flight booking.
- It maintains information about the current conversation, such as destination, date, time, and airline preference.
- It uses the available flight information to decide what response should be given.
- Therefore, it is more advanced than a simple reflex agent because its response depends on the conversation state and available flight information.

(c) Component Responsible for Each Response

Response 1:

"Sure! What time would you like to depart, and do you have a preferred airline?"

- NLU: Extracts destination = Chennai and date = next Friday.
- Dialogue Manager: Detects that departure time and airline preference are missing.
- NLG: Generates the natural-language question asking for these details.

Response 2:

"I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?"

- NLU: Interprets "Morning flight, no preference."
- Knowledge Base/Flight Database: Provides matching flight availability.
- Dialogue Manager: Filters the results and decides to offer the cheapest option.
- NLG: Converts the decision into the displayed response.

(d) Enhancement Using a Learning Agent

A learning-agent architecture could record previous booking interactions and user choices. For example, if a user repeatedly chooses low-cost flights, the system can learn this preference and prioritize cheaper flights in future searches. A feedback mechanism can measure whether the user accepts or rejects recommendations, allowing the chatbot's recommendation strategy to improve over time.

Q2. Real-Time Emotion Detection System

Since the question refers to a processing pipeline diagram but the actual diagram is not visible in the text, the standard pipeline can be reverse-engineered as follows:

Input → Preprocessing → Feature Extraction → Emotion Classification → Decision/Output

(a) Function at Each Stage

Stage	Function
Input Capture	Captures a live image/video or audio signal from the user.
Preprocessing	Removes noise, resizes the input, normalizes intensity, and prepares the data for analysis.
Feature Extraction	Extracts useful facial or audio characteristics such as eyes, mouth, facial landmarks, or voice features.
Emotion Classification	Uses the extracted features to identify emotions such as happy, sad, angry, surprised, or neutral.
Output/Decision	Displays the detected emotion or sends it to another application for real-time action.

b) Likely AI/ML Technique

- **Input Capture:** Camera/microphone and real-time streaming.
- **Preprocessing:** Image normalization, resizing, noise filtering, and face detection.
- **Feature Extraction:** CNN-based feature extraction or facial landmark detection.
- **Emotion Classification:** CNN, SVM, Random Forest, or another supervised classification model.
- **Decision:** Highest-probability emotion is selected from the model output.

For a modern vision-based system, a **CNN/deep-learning model** is a likely choice because it can automatically learn facial patterns associated with different emotions.

C) PEAS Description

PEAS Component	Description
Performance Measure	High emotion-classification accuracy, low latency, stable real-time detection, and low false predictions.
Environment	Real-world users, camera feed, lighting conditions, background, facial expressions, and surrounding noise.
Actuators	Displays detected emotion, generates alerts, or triggers an application response.
Sensors	Camera and/or microphone capturing facial expressions and speech/audio.

(d) Agent Type

The system can be classified primarily as a **model-based agent**.

It continuously observes the user's facial/audio input and maintains information about the current observed state. It does not simply react to one fixed condition; it processes changing input frames and uses a learned model to infer the user's emotional state.

If the system only applies fixed rules such as "*if smile detected* $\rightarrow$ *happy*", it would resemble a **simple reflex agent**. However, a trained ML emotion classifier is better described as a **model-based intelligent agent** because it interprets observations using an internal learned representation.

Q3. Medical Expert System Shell

(a) Type of Chaining

The system uses **forward chaining**.

The reasoning starts with the **known facts** and moves toward a conclusion:

Initial facts:

- fever = yes
- cough = yes
- body_ache = yes

Step 1:

R1 checks fever and cough.

fever = yes AND cough = yes $\rightarrow$ suspect = flu

Therefore:

suspect = flu

Step 2:

R2 checks suspect = flu and body_ache = yes.

flu + body_ache $\rightarrow$ diagnosis = Influenza

Therefore:

Diagnosis = Influenza

This is forward chaining because the inference proceeds **from facts** $\rightarrow$ **intermediate conclusion** $\rightarrow$ **final diagnosis**.

(b) Expert-System Components

Component	Role in the Given System
Knowledge Base	Contains R1, R2, R3 and represents the medical knowledge as IF–THEN rules.
Inference Engine	Matches the given facts against rules and fires R1 followed by R2.
Explanation Facility	Generates the explanation showing why Influenza was selected.
Working Memory	Stores current facts such as fever, cough, body ache, and the newly inferred suspect = flu.

The given explanation:

"Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza."

is produced by the **explanation facility** using the inference trace.

(c) Knowledge Acquisition for a Dengue Rule

A medical expert would provide the required symptoms and diagnostic relationship to the knowledge engineer.

For example, if the expert establishes:

IF fever = yes AND rash = yes AND body_pain = yes THEN diagnosis = Dengue

the knowledge acquisition process would be:

1. Obtain the diagnostic knowledge from a medical expert.
2. Identify the relevant symptoms and conditions.
3. Convert the knowledge into an IF–THEN rule.

4. Validate the rule with the domain expert.
5. Add the rule to the knowledge base.
6. Test it using suitable patient facts.

The new rule then becomes available to the inference engine during future diagnosis.

(d) Could Backward Chaining Reach the Same Diagnosis?

Yes.

Backward chaining would start with the goal:

Goal: diagnosis = Influenza

The inference engine searches for a rule that produces this conclusion. It finds **R2**:

IF suspect = flu AND body ache = yes $\rightarrow$ diagnosis = Influenza

It then asks whether:

- suspect = flu is true.
- body_ache = yes is true.

body_ache = yes is already a given fact.

To prove suspect = flu, it examines R1:

fever = yes AND cough = yes $\rightarrow$ suspect = flu

Both facts are available, so R1 proves suspect = flu.

Therefore, backward chaining can also reach:

diagnosis = Influenza

The difference is that **forward chaining starts from facts**, whereas **backward chaining starts from the desired goal**.

Q4. Autonomous Drone for Disaster Detection with Edge AI

(a) PEAS Description

PEAS Component	Description
Performance Measure	Accurate disaster detection, safe navigation, obstacle avoidance, low response time, efficient battery usage, and successful area coverage.
Environment	Disaster-affected areas containing buildings, roads, obstacles, smoke, fire, people, damaged structures, and changing weather conditions.
Actuators	Propellers/motors, flight-control mechanisms, camera orientation, warning systems, and communication module.
Sensors	RGB/thermal cameras, GPS, IMU, LiDAR/ultrasonic sensors, and other onboard sensors.

(b) Agent Type: Goal-Based or Utility-Based?

The drone is best classified as a **utility-based agent**.

Its goal is not merely to reach a destination. It must balance several competing objectives:

- detect disaster events accurately,
- avoid obstacles,
- conserve battery,
- cover the maximum useful area,
- maintain safe flight,
- communicate important information.

Therefore, the drone can assign different utilities to possible actions and select the action that provides the **best overall outcome**.

(c) Edge vs Cloud Components

Edge/On-Drone Processing:

- Camera/image acquisition
- Preprocessing
- Object/disaster detection
- Obstacle detection
- Immediate navigation and flight control
- Emergency decision-making

Cloud Processing:

- Long-term data storage
- Large-scale analysis
- Model retraining
- Historical disaster analysis
- Generation of detailed reports/maps
- Coordination of information from multiple drones

Why Edge AI Is Used

On-edge processing is important because obstacle avoidance and emergency detection require **very low latency**.

If every camera frame had to be sent to the cloud before the drone could react, network delay or loss of connectivity could cause unsafe flight. Processing directly on the drone allows it to detect obstacles and disasters immediately, even when internet connectivity is poor.

Cloud processing is therefore better suited for **non-critical, computationally intensive tasks**, while safety-critical decisions remain on the edge.

(d) Control Architecture

The drone uses a **hybrid architecture** combining reactive and deliberative control.

Reactive layer:

Immediately responds to obstacles, sudden hazards, and emergency situations.

Example:

Obstacle detected → change direction immediately

Deliberative layer:

Plans routes, selects disaster-search areas, determines coverage strategy, and considers longer-term objectives.

Thus:

Sensors → Reactive safety control + Deliberative planning → Flight control

The hybrid architecture is suitable because the drone needs both **fast responses** and **long-term planning**.

Q5. Deepfake & Face Authenticity Verification

(a) Likely Input Features

The system appears to use multiple facial/video characteristics.

1. Blink Rate

The number of eye blinks over a fixed time period can help identify unnatural facial behavior.

2. Texture/Artifact Score

This can represent visual inconsistencies such as:

- unnatural skin texture,
- blending artifacts,
- abnormal edges,
- compression inconsistencies,
- face-generation artifacts.

Other possible features include facial landmarks, eye movement, lip synchronization, lighting consistency, head movement, and temporal changes between frames.

(b) Likely Model/Technique

The output is likely produced by a **binary classification model** that estimates the probability that a frame is fake.

A deep-learning implementation could use:

- CNN for spatial facial features,
- CNN + LSTM/Transformer for temporal video information,
- an ensemble of multiple classifiers.

The output can be interpreted as:

Fake Confidence = probability that the input is Fake.

For example:

- 8% → probably Real
- 74% → likely Fake
- 91% → strongly Fake

(c) Deriving the Final Real/Fake Decision

A threshold can be applied to the frame-level fake confidence.

For example, using:

Fake confidence $\geq 50\%$ → Fake

Fake confidence $< 50\%$ → Real

The five frames give:

Frame	Fake Confidence	Frame Prediction
1	8%	Real
2	74%	Fake
3	91%	Fake
4	5%	Real
5	68%	Fake

Therefore:

- Real frames = 2
- Fake frames = 3

Using majority voting, the overall video would be classified as:

Final Prediction = Fake

A more robust system could calculate a weighted average of frame-level probabilities rather than simply counting predictions.

The average fake confidence here is:

$$(8 + 74 + 91 + 5 + 68) / 5 = 49.2\%$$

So a simple 50% threshold on the average would actually produce **Real**, showing why majority voting and probability aggregation can produce different results. In practice, the system should define its aggregation method during validation rather than assuming one rule.

(d) Swarm/Ensemble-Based Improvement

A **swarm/ensemble approach** could use several independent detectors instead of relying on one model.

For example:

- Model 1 → facial texture analysis
- Model 2 → blink/eye behavior analysis
- Model 3 → temporal consistency analysis
- Model 4 → CNN-based artifact detection

Their outputs can be combined using **weighted voting or confidence averaging**.

If one detector incorrectly classifies a frame because of poor lighting, the other models can compensate. This makes the final Real/Fake decision **more robust and reliable** than depending on a single model.